НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ «МЭИ»
Институт автоматики и вычислительной техники (АВТИ)
Кафедра управления и интеллектуальных технологий (УИТ)

ПОЛНОЕ ОБЪЯСНЕНИЕ И ПОСТРОЧНЫЙ РАЗБОР ПРОГРАММНОГО КОДА К ЛАБОРАТОРНОЙ РАБОТЕ № 1 ПО ДИСЦИПЛИНЕ «УБС»

Тема: «Структурный анализ больших систем управления. Поуровневое упорядочение и метод Беллмана»
Бригада № 3 (Вариант № 3): студенты Грудинин Е.К., Турханов А.К. (гр. А-03-23)
Преподаватель: ассистент Обычайко Д.С. | Москва, 2026 г.

Настоящий документ содержит исчерпывающее методическое и построчное описание программного кода лабораторной работы №1. Программный комплекс выполняет формализованное математическое описание графа документооборота, разбиение на уровни иерархии, топологическую перенумерацию с формированием строго верхнетреугольной матрицы и пошаговый расчет минимального пути методом динамического программирования (уравнение Беллмана).

1. Функция get_variant_3(): формирование графа Варианта 3

Назначение функции: Формирует размерность информационной системы ($n = 17$) и исходную матрицу смежности/цен дуг A с учетом базового графа методических указаний и двух индивидуальных дуг Варианта 3.

```
 1: def get_variant_3():
 2:     n = 17
 3:     mat = [[0] * n for _ in range(n)]
 4:     base_edges = [
 5:         (1, 9, 2),  (2, 17, 2), (3, 5, 4),  (3, 7, 5),  (3, 11, 3),
 6:         (4, 9, 2),  (5, 4, 3),  (5, 6, 4),  (6, 12, 3), (7, 1, 2),
 7:         (7, 4, 2),  (8, 14, 2), (9, 16, 3), (10, 15, 1), (11, 1, 3),
 8:         (11, 2, 4), (12, 16, 2),(13, 15, 2),(14, 15, 2), (16, 10, 3),
 9:         (16, 13, 3),(17, 8, 2),
10:         # Дополнительные дуги Варианта 3
11:         (11, 4, 5), (6, 17, 5)
12:     ]
13:     for u, v, w in base_edges:
14:         mat[u - 1][v - 1] = w
15:     return n, mat
```

Построчный разбор:

• **Строка 2 (n = 17):** фиксирует порядок графа (общее количество событий/документов в системе).

• **Строка 3 (mat = [[0] * n for _ in range(n)]):** создает двумерный массив (матрицу смежности) размером 17×17, изначально заполненный нулями (отсутствие связей).

> **Важная особенность языка Python:**
> Конструкция [[0] * n] * n ошибочна, так как размножает ссылки на один и тот же подсписок (изменение одной ячейки поменяло бы весь столбец). Списковое включение [for _ in range(n)] создает 17 независимых строк в памяти компьютера.

• **Строки 4–12 (base_edges = [...]):** список направленных дуг графа в формате кортежей (u, v, w), где u — начальная вершина, v — конечная вершина, w — вес дуги (время подготовки документа в условных единицах).

• • **Строки 5–9:** 22 базовые технологические связи, единые для всех вариантов группы.

• • **Строка 11:** индивидуальные дуги Варианта 3 по журналу: дуга (11 → 4) весом 5 и дуга (6 → 17) весом 5.

• **Строки 13–14 (for u, v, w in base_edges: mat[u - 1][v - 1] = w):** цикл заполнения матрицы. Вычитание единицы (u - 1, v - 1) обеспечивает корректное преобразование из математической нумерации вершин (1..17) в системные индексы массивов Python (0..16).

• **Строка 15 (return n, mat):** функция возвращает кортеж из двух значений: размерность и заполненную матрицу смежности.

2. Функция compute_levels(n, mat): поуровневое упорядочение графа

Назначение функции: Реализует алгоритм построения порядковой функции. Проверяет граф на ацикличность, разделяет множество вершин на непересекающиеся иерархические уровни $N_0, N_1, \dots, N_6$, формирует новую сквозную нумерацию и строит каноническую верхнетреугольную матрицу.

```
 1: def compute_levels(n, mat):
 2:     in_nodes = {j: {i for i in range(n) if mat[i][j] > 0} for j in range(n)}
 3:
 4:     levels = []
 5:     visited = set()
 6:     current_level = {j for j, preds in in_nodes.items() if not preds}
 7:
 8:     while current_level:
 9:         levels.append(sorted(current_level))
10:         visited.update(current_level)
11:
12:         next_level = {
13:             j for j in range(n)
14:             if j not in visited and in_nodes[j].issubset(visited)
15:         }
16:         current_level = next_level
17:
18:     if len(visited) != n:
19:         missing = sorted(v + 1 for v in set(range(n)) - visited)
20:         raise ValueError(f"В графе обнаружен контур (цикл)! Невозможно упорядочить вершины:
{missing}")
21:
22:     levels_1based = [[v + 1 for v in lvl] for lvl in levels]
23:
24:     old_to_new = {}
25:     new_to_old = {}
26:     new_num = 1
27:     for lvl in levels_1based:
28:         for old_v in lvl:
29:             old_to_new[old_v] = new_num
30:             new_to_old[new_num] = old_v
31:             new_num += 1
32:
33:     ordered_mat = [[0] * n for _ in range(n)]
34:     for i in range(1, n + 1):
35:         for j in range(1, n + 1):
36:             ordered_mat[i - 1][j - 1] = mat[new_to_old[i] - 1][new_to_old[j] - 1]
```

```
37:
38:     return levels_1based, old_to_new, new_to_old, ordered_mat
```

Построчный разбор:

• **Строка 2:** предвычисление множеств левых инциденций $G^{-1}(j)$ за один проход $O(n^2)$. Для каждого столбца j отбираются строки i, где есть связь mat[i][j] > 0. Это устраняет необходимость многократного сканирования матрицы внутри циклов.

• **Строки 4–6:** инициализация списка уровней levels, множества посещенных вершин visited и нулевого уровня current_level (вершины с пустым списком предков — истоки системы №, в нашем графе это вершина 3).

• **Строка 8 (while current_level:):** цикл продолжается до тех пор, пока удается сформировать следующий непустой иерархический уровень.

• **Строки 9–10:** текущий уровень упорядочивается и добавляется в общий список уровней, а все входящие в него вершины помечаются как пройденные (visited.update).

• **Строки 12–16 (next_level = {...}):** теоретико-множественное формирование следующего уровня:

> **Математический смысл алгоритма уровней:**
> Вершина j включается в уровень N_m тогда и только тогда, когда:
> 1. Она еще не была распределена ранее (j not in visited);
> 2. ВСЕ её непосредственные предшественники уже включены в предыдущие уровни:
> in_nodes[j].issubset(visited) <=> G^-1(j) ⊆ (N_0 ∪ N_1 ∪ ... ∪ N_{m-1}).

• **Строки 18–20:** проверка условия ацикличности графа (|visited| = n). Если в графе есть замкнутый контур, вершины цикла никогда не удовлетворят условию issubset, и цикл завершится преждевременно. Программа выбрасывает исключение с перечнем заблокированных вершин.

• **Строка 22:** преобразование системных индексов 0..16 обратно в исходную нумерацию вершин 1..17.

• **Строки 24–31:** поуровневое присвоение вершинам новых порядковых номеров 1..17. Формируются словари прямой (old_to_new) и обратной (new_to_old) перенумерации.

• **Строки 33–36:** построение упорядоченной матрицы смежности. За счет топологического порядка для любой дуги номер источника строго меньше номера стока ($i < j$). Поэтому все элементы на главной диагонали и ниже неё равны нулю ($a_{ij} = 0$ при $i \geq j$), что математически доказывает строго верхнетреугольный вид матрицы.

• **Строка 38:** возврат четырехуровневого результата: уровни, два словаря нумерации и треугольная матрица.

3. Функция compute_bellman_path(...): расчет методом Беллмана

Назначение функции: Реализует метод динамического программирования на упорядоченном графе без контуров на основе функционального уравнения Беллмана. Рассчитывает минимальную длину пути, формирует детальный математический протокол и восстанавливает все альтернативные маршруты.

```
1: def compute_bellman_path(n, ordered_mat, new_to_old, old_to_new, start_old=3, end_old=15):
```

```
    start_new = old_to_new.get(start_old, 1)
    end_new = old_to_new.get(end_old, n)

    f_min = {i: float('inf') for i in range(1, n + 1)}
    parents = {i: [] for i in range(1, n + 1)}
    f_min[start_new] = 0

    steps_log = []

    for new_j in range(start_new, end_new + 1):
        old_j = new_to_old[new_j]
        incoming = []

        for new_i in range(start_new, new_j):
            w = ordered_mat[new_i - 1][new_j - 1]
            prev_cost = f_min[new_i]
            if w > 0 and prev_cost != float('inf'):
                incoming.append((new_i, new_to_old[new_i], w, prev_cost + w))

        if incoming:
            best_cost = min(item[3] for item in incoming)
            f_min[new_j] = best_cost
            parents[new_j] = [item[0] for item in incoming if item[3] == best_cost]

            variants = [
                f"q({old_i},{old_j})+F({old_i})={w}+{f_min[new_i]}={cand}"
                for new_i, old_i, w, cand in incoming
            ]
            steps_log.append(f"Вершина #{new_j} (исх. {old_j}): min({', '.join(variants)}) =
{best_cost}")
        elif new_j == start_new:
            steps_log.append(f"Вершина #{new_j} (исх. {old_j}): источник, F({old_j}) = 0")

    if f_min[end_new] == float('inf'):
        return None, [], [], steps_log

    paths_new = []

    def backtrack(curr, current_path):
        if curr == start_new:
            paths_new.append(list(reversed(current_path)))
            return
        for p in parents[curr]:
            backtrack(p, current_path + [p])

    backtrack(end_new, [end_new])
    paths_old = [[new_to_old[v] for v in p] for p in paths_new]

    return f_min[end_new], paths_old, paths_new, steps_log
```

Построчный разбор:

• **Строки 2–3:** перевод исходных номеров источника (3) и стока (15) в новые топологические индексы (#1 и #17).

• **Строки 5–7:** инициализация оценок Беллмана. Для источника f_min[start_new] = 0, для всех остальных вершин задается плюс-бесконечность (inf). Словарь parents хранит списки оптимальных предков для ветвления альтернативных путей.

• **Строка 11 (for new_j in range(start_new, end_new + 1):):** прямой топологический проход по вершинам от начала к концу. Гарантирует, что к моменту рассмотрения вершины new_j все возможные предки уже окончательно рассчитаны.

• **Строки 15–19:** сбор всех входящих дуг. Благодаря верхнетреугольной матрице все потенциальные предшественники имеют номер new_i < new_j. Кандидатный вес пути рассчитывается как сумма оценки предка и веса дуги (prev_cost + w).

• **Строки 21–24:** реализация принципа оптимальности Беллмана. Находится минимум из всех входящих значений: $F(j) = min_{i \in G^{-1}(j)}[F(i) + q(i,j)]$. Все предки, давшие минимальное значение, сохраняются в список parents[new_j], что обеспечивает сохранение параллельных путей.

• **Строки 26–32:** формирование подробной символьной записи математического шага для GUI и отчета (например, q(7,1)+F(7)=2+5=7, q(11,1)+F(11)=3+3=6 => min = 6).

• **Строки 34–35:** контроль связности графа. Если до конечной вершины добраться невозможно, возвращается None.

• **Строки 37–45:** рекурсивная функция обратного хода (backtrack). Спускается от стока к источнику по ссылкам parents. При наличии развилок с одинаковой стоимостью рекурсия обходит каждую ветвь, находя все альтернативные маршруты. Достигнув истока, разворачивает накопленный путь в прямой порядок.

• **Строка 46:** преобразование маршрутов из топологической нумерации обратно в исходные номера события документооборота.

• **Строка 48:** возврат итоговой минимальной стоимости L_min, списков маршрутов в обеих нумерациях и протокола вычислений.

4. Вопросы преподавателя к защите лабораторной работы

Вопрос 1: В чем отличие метода Беллмана от полного перебора (Brute Force)?

Ответ: Полный перебор находит ВСЕ мыслимые пути от источника к стоку и выбирает из них минимальный. Его трудоемкость растет экспоненциально (O(2^V) или O(V!)), и на графе из 50–100 вершин программа зависнет навсегда. Метод Беллмана опирается на принцип динамического программирования: хвост оптимального пути сам является оптимальным. Нам не нужно помнить предысторию от старта — достаточно знать оптимальную оценку F(i) непосредственных предшественников и добавить вес связывающей дуги. Сложность метода Беллмана на DAG линейна: O(V + E), расчет выполняется за доли миллисекунды.

Вопрос 2: Зачем граф предварительно разбивается на уровни?

Ответ: В произвольном графе с контурами алгоритм Беллмана требует множественных итераций релаксации (до V - 1 проходов). Поуровневое упорядочение графа без контуров гарантирует, что дуги направлены строго с меньших уровней на большие. Поэтому, перебирая вершины в топологическом порядке (по уровням от N_0 до N_6), мы уверены: к моменту расчета вершины j оптимальные оценки F(i) ДЛЯ ВСЕХ её предшественников уже окончательно вычислены. Это сводит решение всей задачи ровно к одному прямому проходу.

Вопрос 3: Как доказать, что в графе нет контуров и матрица треугольная?

Ответ: Отсутствие контуров доказывается успешным распределением всех 17 вершин по уровням (|visited| = n). Если бы был контур A -> B -> A, эти вершины никогда бы не удовлетворили условию замыкания предков. Треугольный вид матрицы означает, что при новой сквозной нумерации все ненулевые элементы a_ij расположены строго выше главной диагонали (i < j), а на диагонали и под ней строго нули (a_ij = 0 при i >= j).

Вопрос 4: Каковы численные результаты решения для Варианта 3?

Ответ: Минимальное время прохождения документов составляет $L_{min} = 15$. В системе существуют ровно два равноценных оптимальных маршрута:
1) 3 -> 11 -> 1 -> 9 -> 16 -> 10 -> 15 (время: 3 + 3 + 2 + 3 + 3 + 1 = 15);
2) 3 -> 11 -> 2 -> 17 -> 8 -> 14 -> 15 (время: 3 + 4 + 2 + 2 + 2 + 2 = 15).
Критический (максимальный) путь: 3 -> 5 -> 6 -> 17 -> 8 -> 14 -> 15 (время: 4 + 4 + 5 + 2 + 2 + 2 = 19).